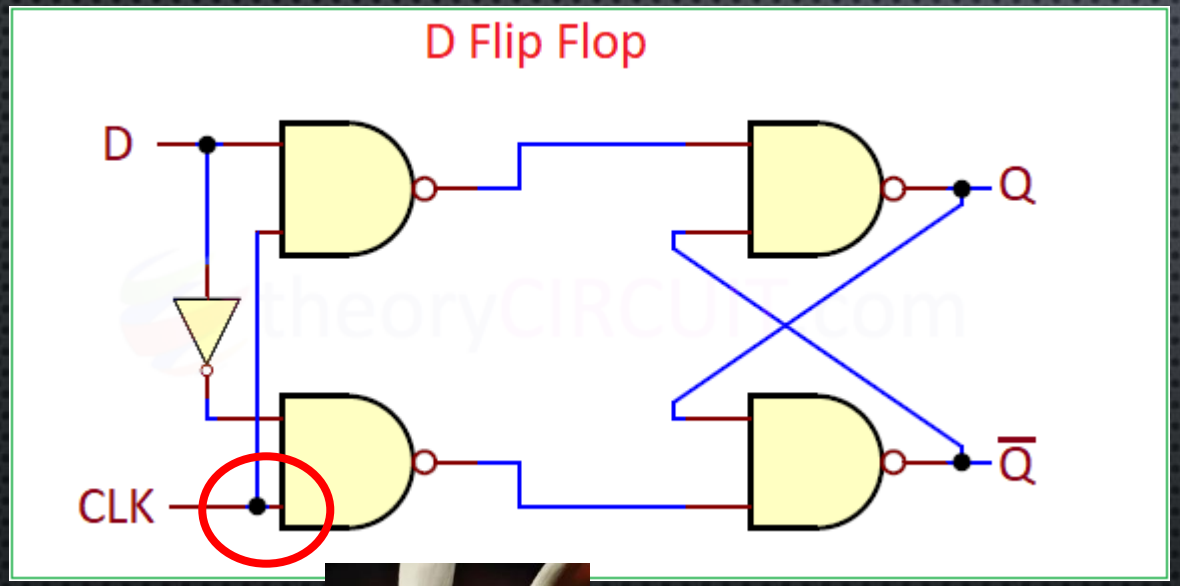
DIGITAL SYSTEMS

DESIGN AND VERIFICATION

D-FLIP FLOPS



CLK	O	Q	$\overline{Q}$
0	0	Q	$\overline{Q}$
0	1	Q	$\overline{Q}$
1	0	0	1
1	1	1	0

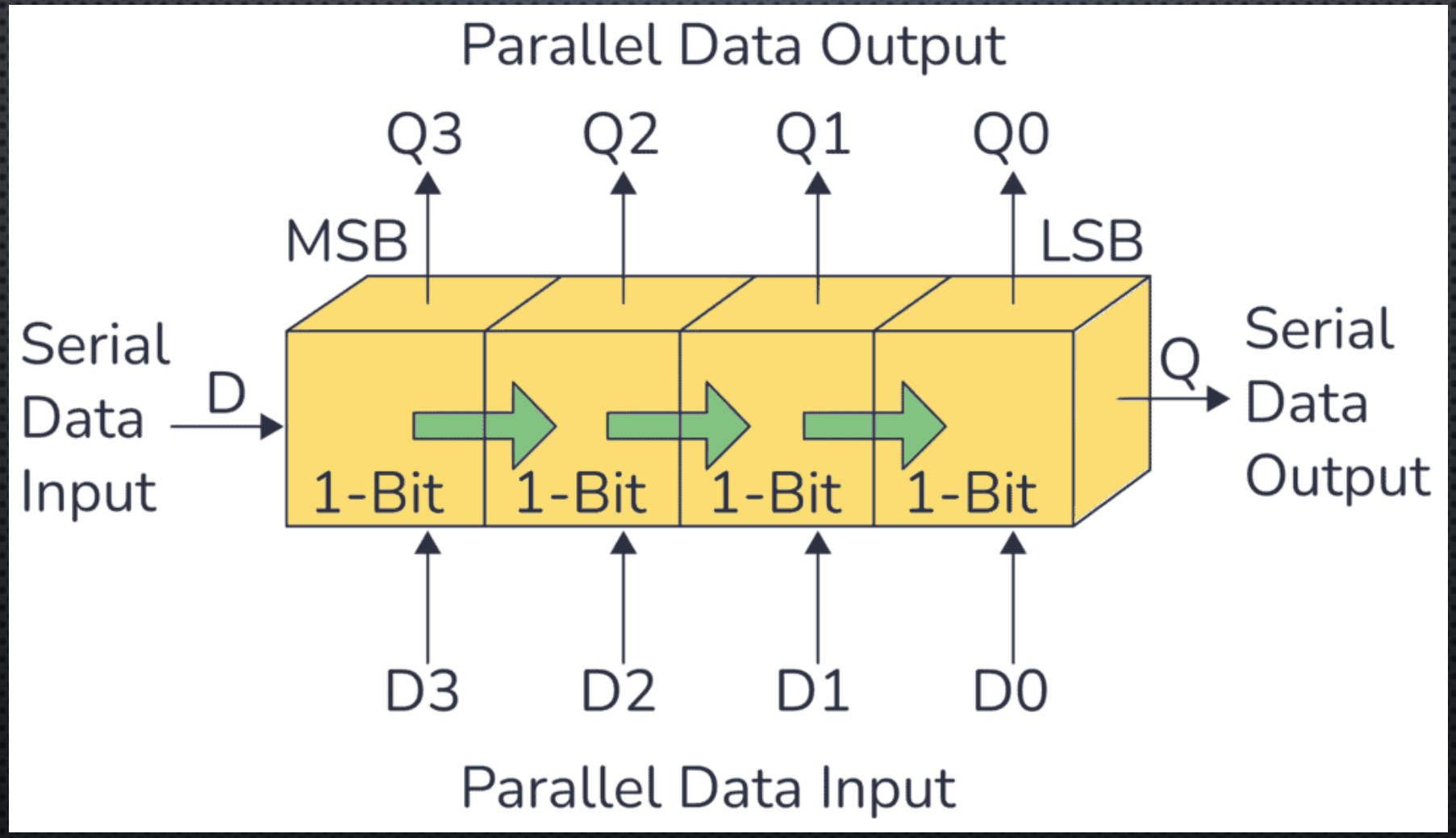
D-FLIP FLOPS

Key Applications:

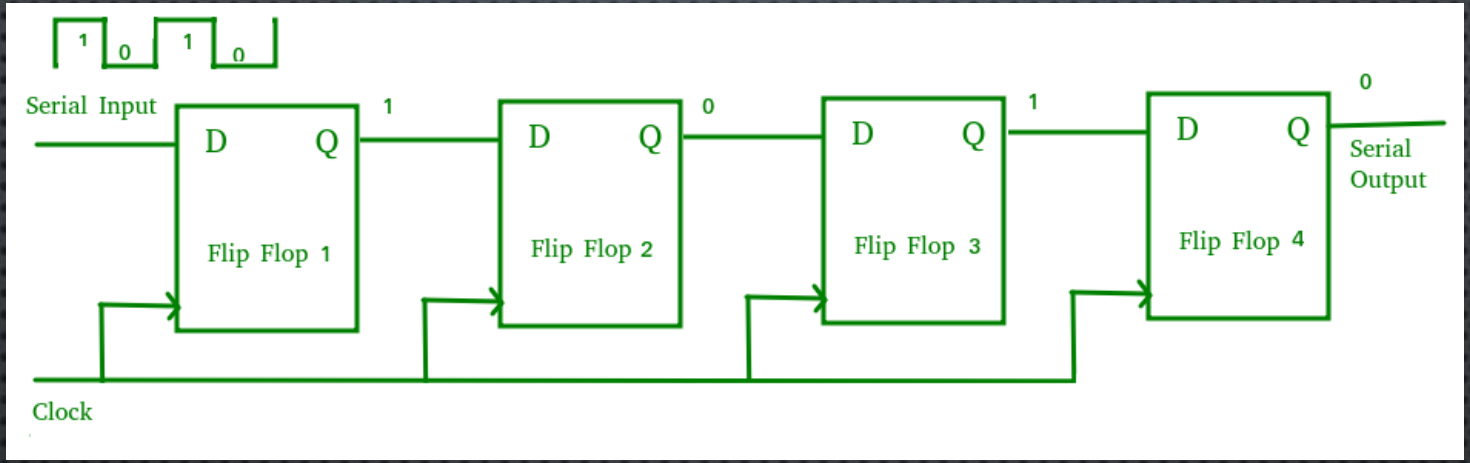
- **Data Storage:** Used to build RAMs, registers and digital information.
- **Shift Register:** By connecting them in series you make shift registers, to convert data from serial to parallel, and from parallel to serial
- **Counters:** Used in freq div circuits
- **Synchronization:** Allign received data to a specific clock domain
- **State Machines:** Store the current state and get next state

```
module d_flip_flop (
    input  wire clk,
    input  wire reset_n,  // Active-low asynchronous reset
    input  wire d,
    output reg  q
);


    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            q <= 1'b0;    // Clear output when reset is 0
        end else begin
            q <= d;       // Capture input on the clk edge
        end
    end

endmodule
```

SHIFT REGS


Parallel Data Output
Q3
Q2
Q1
Q0
MSB
LSB
Serial
Data
Input
D
Q
Serial
Data
Output
1-Bit
1-Bit
1-Bit
1-Bit
D3
D2
D1
D0
Parallel Data Input

SHIFT REGS



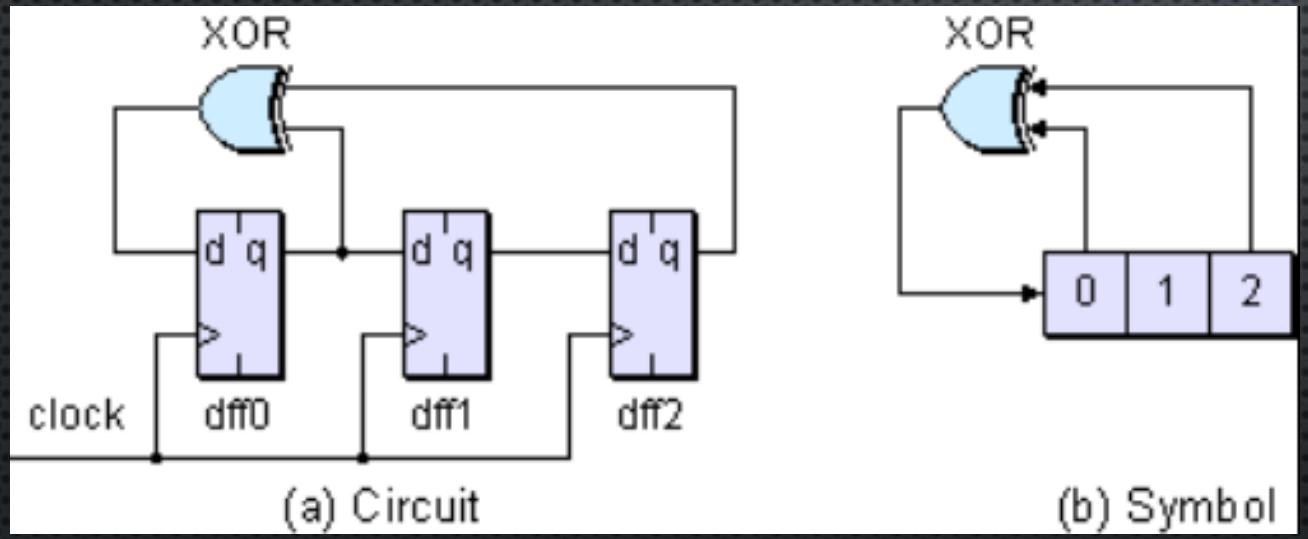
Key Applications:

- **Types:** SISO, SIPO, PISO, PIPO
- **GPIO expansion:** Control/read multiple devices with few pins
- **Counters:** LFSR (pseudo random num generator)
- **Delay Timing:** Delay signal by x clock cycles

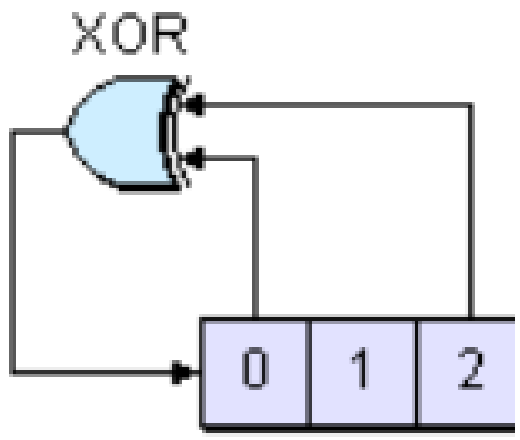
SHIFT REG EXAMPLE

```
module shift_register_4bit (
    input  wire clk,
    input  wire reset_n,     // Active-low asynchronous reset
    input  wire enable,
    input  wire shift_in,    // Serial data input
    output reg  [3:0] q      // 4-bit parallel output
);

    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            q <= 4'b0000;    // Clear the register
        end else if (enable) begin
            // Shift left: Concatenate the lower 3 bits with new input
            q <= {q[2:0], shift_in};
        end
    end

endmodule
```

LINEAR FEEDBACK SHIFT REGISTER



LINEAR FEEDBACK SHIFT REGISTER

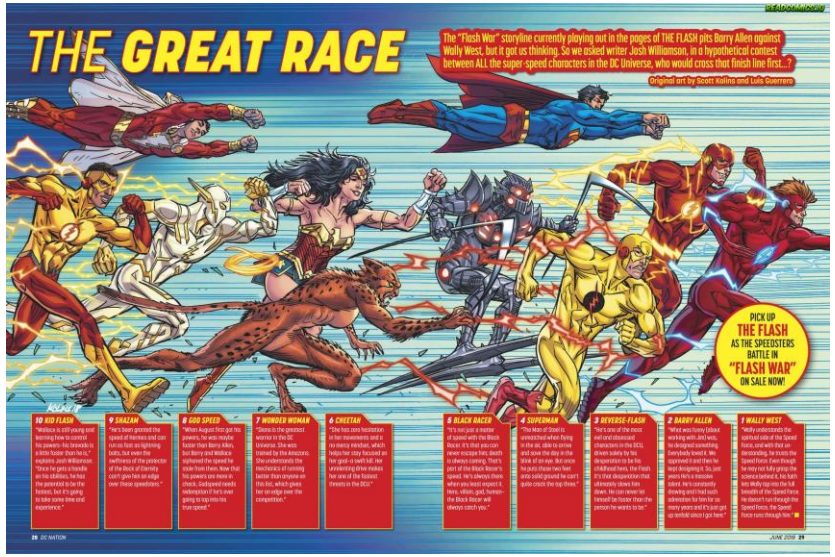


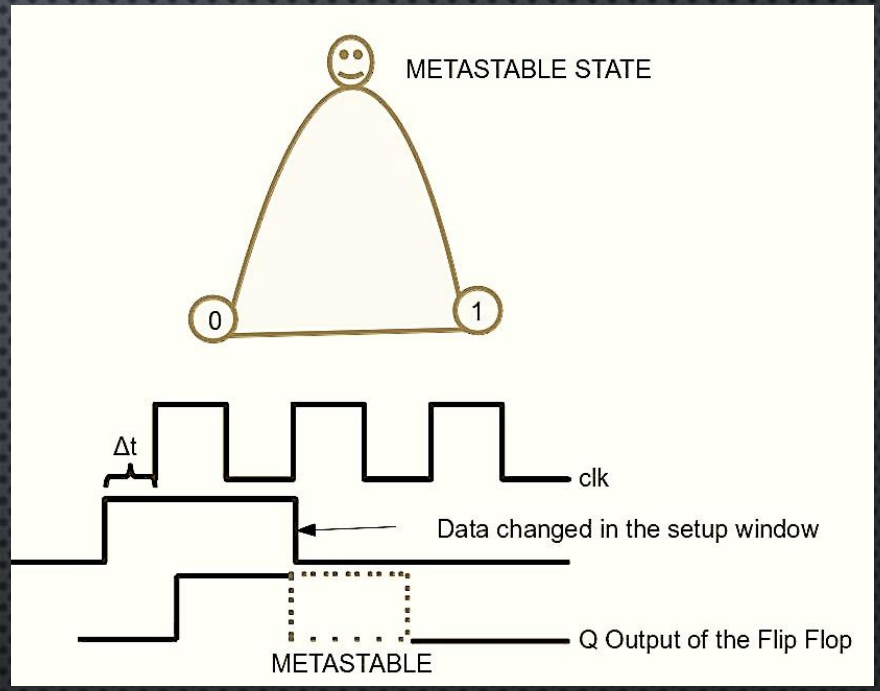

```
module lfsreg #(
    parameter WIDTH = 16,
    parameter [WIDTH-1:0] SEED = 16'h1001,

    // These are the default taps for 16-bit: bits 15, 13, 12, 10
    parameter [WIDTH-1:0] TAP_MASK = 16'hB400
    )(
    input clk, reset_n, enable,
    output reg [WIDTH-1:0] out
    );

    wire feedback;
    assign feedback = ^(out & TAP_MASK);

    always @(posedge clk or negedge reset_n)
    begin
        if(!reset_n)
            out <= SEED;
        else if (enable == 1'b1)
            out <= {out[WIDTH-2:0],feedback};
    end

endmodule
```

CLOCK DOMAIN

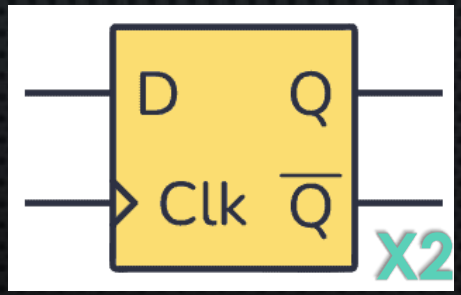
A **clock domain** involves all of the **synchronous** components that are driven by a single **clock**.

Clock Domain Crossing (CDC) occurs whenever a signal generated in one clock domain is used in another clock domain





THE EASY FIX

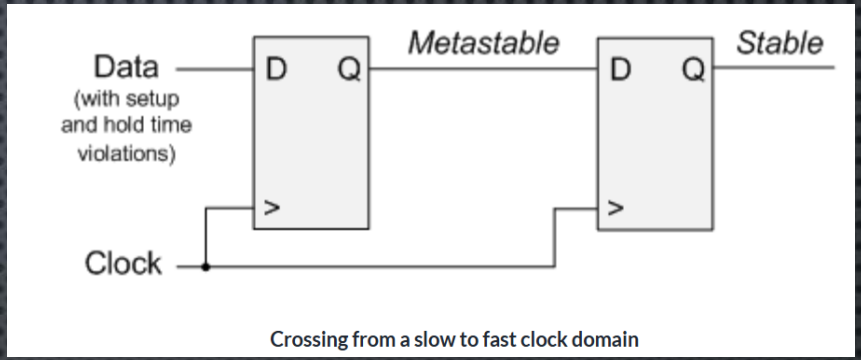


CLOCK DOMAIN – STRATEGIES

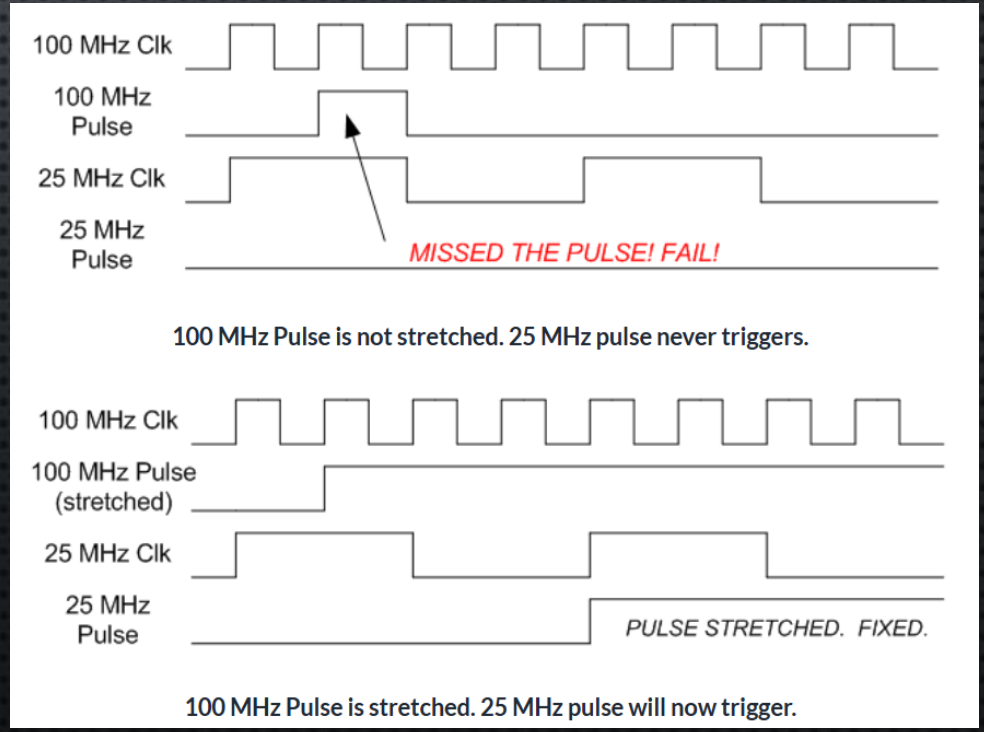
Pro Tips:

- Identify all your clock domains
- Review that every wire that crosses a clock domain has a sync strategy
- Write Timing Constraints

From faster clock domain to slower clock domain → stretch out your signal



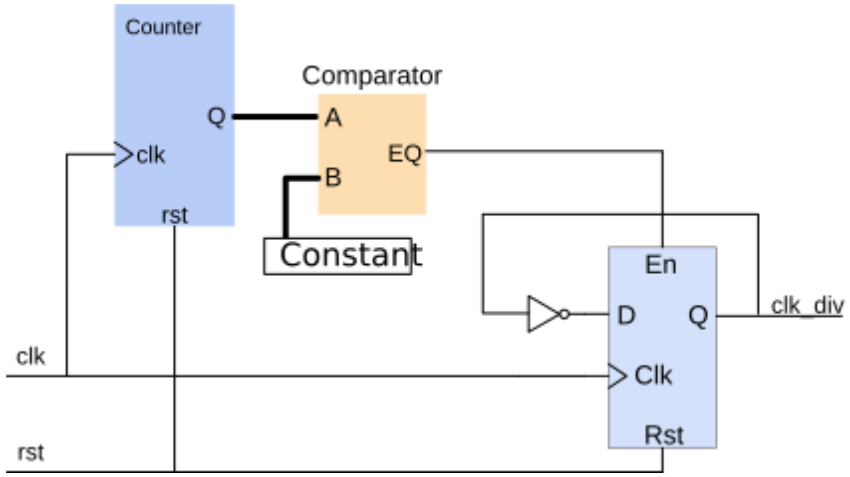
Crossing from a slow to fast clock domain

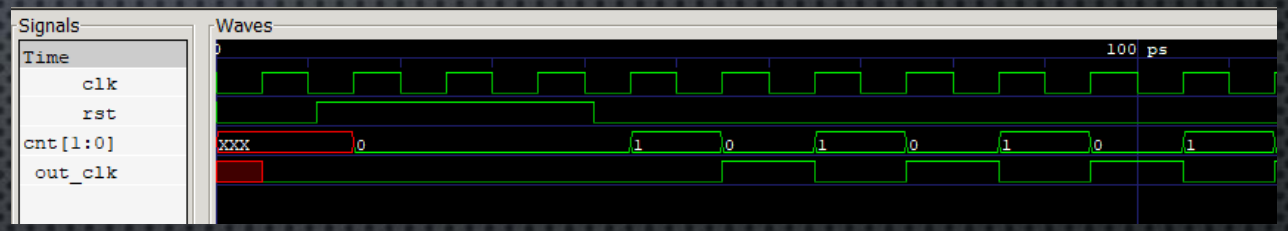


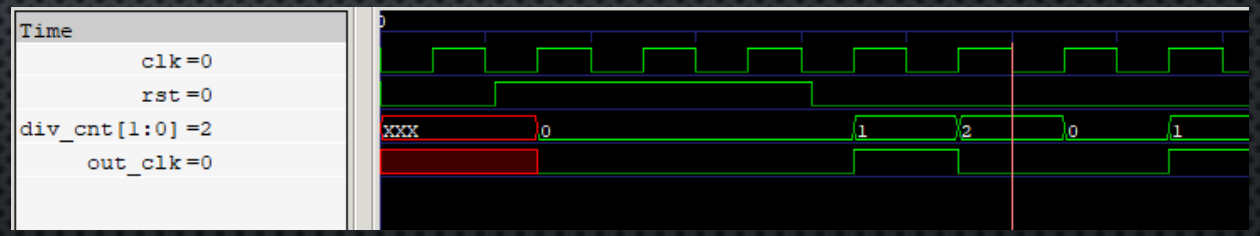
CLOCK DIVIDERS AND TIMING ANALYZER !!

Dividers allow us to blink LEDs, read physical switches, or multiplex 7-segment displays at perceptible speeds

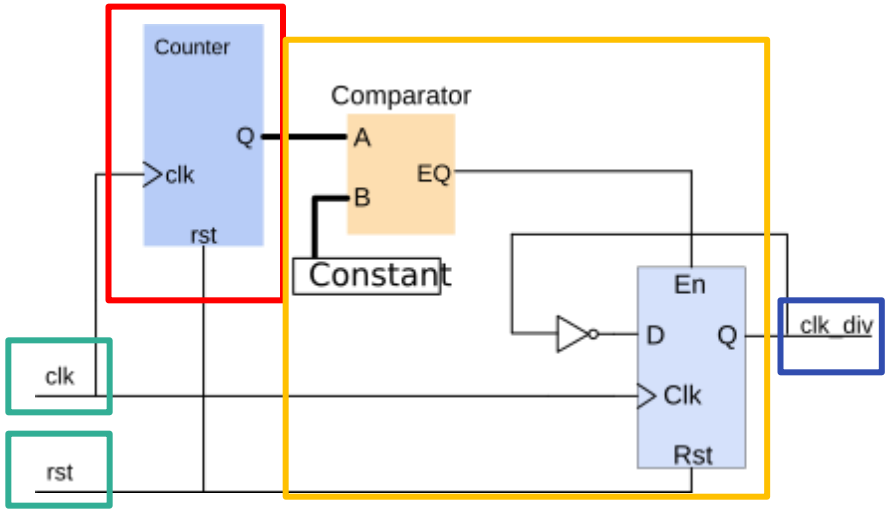
Ex: A slower clock helps sample the button state only after it has stabilized. (Avoiding Bouncing)







CLOCK DIVIDERS !!




```
module ClkDivider (
    input clk,
    input rst,
    output reg clk_div
    );

 localparam constantNumber = 50000000;

endmodule
```

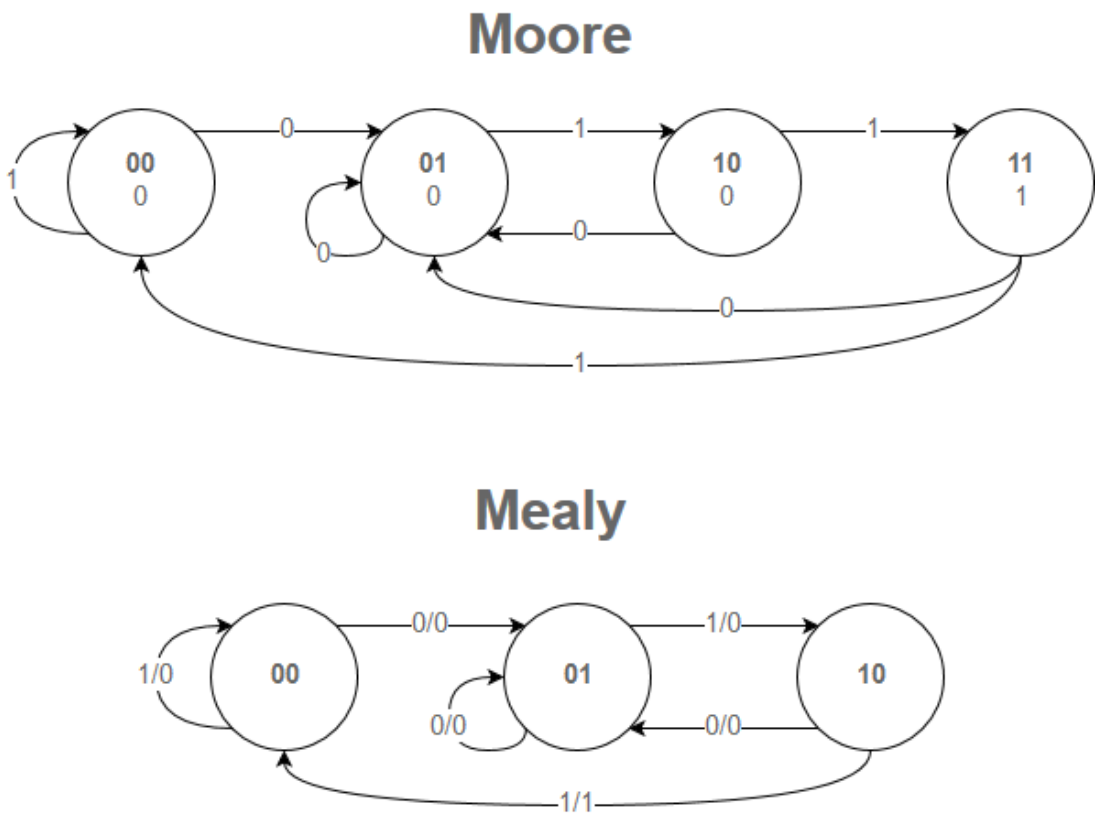
```
reg [31:0] count;

always @ (posedge(clk), posedge(rst))
begin
    if (rst == 1'b1)
        count <= 32'b0;
    else if (count == constantNumber - 1)
        count <= 32'b0;
    else
        count <= count + 1;
end
```

```
always @ (posedge(clk), posedge(rst))
begin
    if (rst == 1'b1)
        clk_div <= 1'b0;
    else if (count == constNumber - 1)
        clk_div <= ~clk_div;
    else
        clk_div <= clk_div;
end
```

FINITE STATE MACHINES (FSM)

A system that can be in **exactly one of a finite number of states at any given time**. We use them to control the flow of data.

- Moore Machine: The output depends ONLY on the current state.
- Mealy Machine: The output depends on the current state AND the current inputs.



EXAMPLES OF FSM

Moore

Traffic Light Controller

Moore

Digital Clock Display

Moore

Elevator Doors

Mealy

Car Alarm System

Mealy

Vending Machine

Mealy

Keyboard

Moore

Washing Machine

Mealy

Smart Door Lock

FSM WITHIN VERILOG

```
module 011_detector (
    input  wire clk,
    input  wire reset_n,
    input  wire data_in,
    output reg  detected
);
```

```
// 1. State Encoding (Moore)
    parameter S0 = 2'b00; // Got nothing
    parameter S1 = 2'b01; // Got '0'
    parameter S2 = 2'b10; // Got '01'
    parameter S3 = 2'b11; // Got '011'

    reg [1:0] current_state, next_state;
```

```
// 1. State Encoding (Mealy)
    parameter S0 = 2'b00; // Got nothing
    parameter S1 = 2'b01; // Got '0'
    parameter S2 = 2'b10; // Got '01'

    reg [1:0] current_state, next_state;
```

```
// 2. State Register (Sequential)
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) current_state <= S0;
        else          current_state <= next_state;
    end
```

```
// 2. State Register (Sequential)
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) current_state <= S0;
        else          current_state <= next_state;
    end
```

```
// 3. Next State Logic (Combinational)
    always @(*) begin
        case (current_state)
            S0: next_state = (data_in) ? S0 : S1;
            S1: next_state = (data_in) ? S2 : S1;
            S2: next_state = (data_in) ? S3 : S1;
            S3: next_state = (data_in) ? S0 : S1;
            default: next_state = S0;
        endcase
    end
```

```
// 4. Output Logic (Depends ONLY on state)
    always @(*) begin
        if (current_state == S3) detected = 1'b1;
        else                     detected = 1'b0;
    end

endmodule
```

```
// 3. Next State Logic (Combinational)
    always @(*) begin
        case (current_state)
            S0: next_state = (data_in) ? S0 : S1;
            S1: next_state = (data_in) ? S2 : S1;
            S2: next_state = (data_in) ? S0 : S1;
            // Goes to S0 on '1' after detection
            default: next_state = S0;
        endcase
    end
```

```
// 4. Output Logic (Depends on state AND data_in)
    always @(*) begin
        // Output is 1 ONLY if we are in S2 AND the incoming bit is 1
        if ((current_state == S2) && (data_in == 1'b1))
            detected = 1'b1;
        else
            detected = 1'b0;
    end

endmodule
```

PROBLEM !!! – DESIGN EXERCISE

We have just installed our 16-bit LFSR and connected the least significant 4 bits (out[3:0]) directly to our Week 2 hex_decoder.

We want to see the random numbers on the physical display of the DE10-Lite. However, when programming the card, the display appears to show a bright '8' all the time.

What went wrong if our Verilog code has no syntax errors?

- Connect the16-bit LFSR to the 7-segment display
  - The numbers must change at exactly 2 Hz (2 times per second)
  - Solve the top level file, compile and run !!